8

# Programação orientada a objetos e a eventos

Universidade Estadual da Paraíba (UEPB)
Paradigmas de Programação
Kláudio Henrique Mascarenhas Medeiros

# Programação orientada a objetos

- O paradigma orientado a objetos busca uma aproximação do mundo real com a modelagem computacional.

# Programação orientada a objetos

- O **paradigma orientado a objetos** busca uma aproximação do mundo real com a modelagem computacional.

- Os elementos e conceitos associados ao problema que está se tratando são **objetos**, com características e operações relacionadas com a entidade real.

# Programação orientada a objetos

- O **paradigma orientado a objetos** busca uma aproximação do mundo real com a modelagem computacional.

- Os elementos e conceitos associados ao problema que está se tratando são **objetos**, com características e operações relacionadas com a entidade real.

- Os sistemas são formados por objetos que **trocam mensagens entre si**, têm atributos e comportamentos.

# Programação orientada a objetos

```java
public class Ponto2D {

    private float x, y;
    public Ponto2D(float x, float y){this.x = x; this.y = y; }
    public float getX() { return this.x; }
    public float getY() { return this.y; }
    public void setX(float novoX) { this.x = novoX; }
    public void setY(float novoY) { this.y = novoY; }
    public double distance(Ponto2D other)
    {
        float difX = this.x - other.x;
        float difY = this.y - other.y;
        return Math.sqrt(Math.pow(difX, 2) + Math.pow(difY, 2));
    }
}
```

# Programação orientada a objetos

```java
public class ClientClass {

    public static void main(String[] args) {
        Ponto2D p1 = new Ponto2D(0, 0);
        Ponto2D p2 = new Ponto2D(4, 2);
        System.out.println(p1.distance(p2));
    }
}
```

- Quem fez a classe ClientClass tem acesso aos métodos públicos de Ponto2D, podendo utilizá-los, mas não necessariamente sabe (ou precisa saber) dos detalhes de implementação desses métodos.

# Programação orientada a objetos

- O paradigma suporta os tipos abstratos de dados:
  - a representação dos valores desses tipos abstratos são **escondidos do usuário** (classes c/ atr. privados);

# Programação orientada a objetos

- O paradigma suporta os tipos abstratos de dados:
  - a representação dos valores desses tipos abstratos são **escondidos do usuário** (classes c/ atr. privados);
  - a **representação do TAD pode ser trocada** sem que usuário precise trocar algo em seus programas;

```java
public class Ponto2D {

    private float x, y;
    public Ponto2D(float x, float y){this.x = x; this.y = y; }
    public float getX() { return this.x; }
    public float getY() { return this.y; }
    public void setX(float novoX) { this.x = novoX; }
    public void setY(float novoY) { this.y = novoY; }
    public double distance(Ponto2D other)
    {
        double exp1 = Math.pow(x - other.x, 2);
        double exp2 = Math.pow(y - other.y, 2);
        return Math.sqrt(exp1 + exp2);
    }
}
```

- Perceba que a implementação de distance mudou, mas ele continua recebendo os mesmos parâmetros e tendo o mesmo retorno. Nada precisa mudar em ClientClass.

# Programação orientada a objetos

- O paradigma suporta os tipos abstratos de dados:
  - a representação dos valores desses tipos abstratos são **escondidos do usuário** (classes c/ atr. privados);
  - a **representação do TAD pode ser trocada** sem que usuário precise trocar algo em seus programas;
  - existem operações **construtoras e destruidoras**, bem como métodos para realização de operações.

```
class Ponto2D
{
    public:
        Ponto2D(float newX, float newY)
        {
            x = newX;
            y = newY;
        }

        ~Ponto2D()
        {
            cout << "Ponto2D(" << getX() << ", " << getY() << ") destruido." << endl;
        }

        void display()
        {
            cout << "Ponto2D(" << x << ", " << y << ")" << endl;
        }
        float getX() const { return this -> x;}
        float getY() const { return y;}
        void setX(float newX) { x = newX;}
        void setY(float newY) { y = newY;}
        void operator +=(int n){
            x += n; y += n;
        }
    private:
```

# Programação orientada a objetos

- O paradigma suporta o conceito de encapsulamento:
  - os atributos e métodos de uma classe podem ter **acesso limitado** (e.g. public, private, protected);

# Programação orientada a objetos

- O paradigma suporta o conceito de encapsulamento:
  - os atributos e métodos de uma classe podem ter **acesso limitado** (e.g. public, private, protected);
  - pode haver também, como em Java, um **encapsulamento lógico**, determinado pela visibilidade de pacote (package).

# Programação orientada a objetos

- O paradigma suporta o conceito de encapsulamento:
  - os atributos e métodos de uma classe podem ter **acesso limitado** (e.g. public, private, protected);
  - pode haver também, como em Java, um **encapsulamento lógico**, determinado pela visibilidade de pacote (package).
  - C++ permite acesso a membros privados por/ funções externas (por exemplo) com **friend**.

# Programação orientada a objetos

- O paradigma envolve classes e objetos:
  - os **objetos** encapsulam o estado de alguma coisa, possuem atributos, acessíveis através de métodos públicos

# Programação orientada a objetos

- O paradigma envolve classes e objetos:
  - os **objetos** encapsulam o estado de alguma coisa, possuem atributos, acessíveis através de métodos públicos
  - a **classe** especifica os atributos e as operações (métodos) de um conjunto desses objetos

# Programação orientada a objetos

- O paradigma envolve classes e objetos:
  - a “mensagem” que falamos em OO diz respeito a uma **chamada de método**, definida pelo objeto ao qual a mensagem é enviada e qual a ação desejada;

```java
public class ClientClass {

    public static void main(String[] args) {
        Ponto2D p1 = new Ponto2D(0, 0);
        Ponto2D p2 = new Ponto2D(4, 2);
        // "p1, calcule sua distância com relação a p2!"
        System.out.println(p1.distance(p2));
    }
}
```

- A mensagem é enviada para p1. O p2 é um argumento do método distance. O resultado é o cálculo da distância em linha reta entre dois pontos, sendo esses pontos p1 e p2.

# Programação orientada a objetos

- O paradigma envolve classes e objetos:
  - a “mensagem” que falamos em OO diz respeito a uma **chamada de método**, definida pelo objeto ao qual a mensagem é enviada e qual a ação desejada;
  - as computações em OO são especificadas pelas **mensagens que são mandadas de e para objetos existentes**.

# Programação orientada a objetos

- O paradigma envolve classes e objetos:
  - as **variáveis podem ser de instância** (cada instância tem seu valor para ela) **ou de classe** (todas as instâncias da classe possuem o mesmo valor);

# Programação orientada a objetos

- O paradigma envolve classes e objetos:
  - as **variáveis podem ser de instância** (cada instância tem seu valor para ela) **ou de classe** (todas as instâncias da classe possuem o mesmo valor);
  - os **métodos podem ser de instância** (aplicado a cada objeto em particular da classe) **ou da classe** (aplicado para toda a classe e não a uma instância em particular).

# Programação orientada a objetos

Em Java, atributos ou métodos de classe possuem a palavra-reservada ***static***. Sem static, entende-se que são de instância

# Programação orientada a objetos

- O paradigma envolve herança:
  - a **herança** é uma forma de criar hierarquias de classes que herdam atributos e comportamentos de outra, bem como estender uma classe c/ algo novo;

# Programação orientada a objetos

Em Java, podemos criar a classe Animal e a classe Gato, herdando de Animal. Dizemos que Gato estende (*extends*) Animal

# Programação orientada a objetos

- O paradigma envolve herança:
  - a **herança** é uma forma de criar hierarquias de classes que herdam atributos e comportamentos de outra, bem como estender uma classe c/ algo novo;
  - os conceitos de subclasse e superclasse;

# Programação orientada a objetos

No exemplo anterior, Gato é subclasse de Animal. Animal é superclasse de Gato.

# Programação orientada a objetos

- O paradigma envolve herança:
  - a **herança** é uma forma de criar hierarquias de classes que herdam atributos e comportamentos de outra, bem como estender uma classe c/ algo novo;
  - os conceitos de subclasse e superclasse;
  - objetos da subclasse podem ser usados no lugar de objetos da superclasse (princípio da substituição).

# Programação orientada a objetos

O **princípio da substituição de Liskov** diz que “se q(x) é uma propriedade demonstrável de objetos x do tipo T, então q(y) deve ser verdadeiro para objetos Y do tipo S, onde S é um subtipo de T”

# Programação orientada a objetos

- O paradigma envolve herança:
  - uma subclasse pode **sobrepor uma implementação** definida na superclasse (override);

# Programação orientada a objetos

- O paradigma envolve herança:
  - uma subclasse pode **sobrepor uma implementação** definida na superclasse (override);
  - pode-se acontecer **herança simples** (uma subclasse herda de apenas uma superclasse) ou **múltipla** (herda de mais de uma superclasse);

# Programação orientada a objetos

- O paradigma envolve herança:
  - uma subclasse pode **sobrepor uma implementação** definida na superclasse (override);
  - pode-se acontecer **herança simples** (uma subclasse herda de apenas uma superclasse) ou **múltipla** (herda de mais de uma superclasse);
  - algumas linguagens permitem apenas a simples.

# Programação orientada a objetos

A herança múltipla pode causar alguns problemas, como o **problema do diamante** (*diamond problem*)

# Programação orientada a objetos

- Ex. Você define a classe Pessoa, com um método que chamaremos de m().

# Programação orientada a objetos

- Ex. Você define a classe Pessoa, com um método que chamaremos de m().

- Você define a classe Pai e a classe Mãe, que herdam de Pessoa (pais e mães são pessoas). Cada classe realiza o override de m() para suas próprias implementações.

# Programação orientada a objetos

- Ex. Você define a classe Pessoa, com um método que chamaremos de m().

- Você define a classe Pai e a classe Mãe, que herdam de Pessoa (pais e mães são pessoas). Cada classe realiza o override de m() para suas próprias implementações.

- Você define a classe Filho, que herda de Pai e de Mãe. Filho herdará traços de Pessoa duas vezes. Caso não faça override, que implementação de m() ele tem?

# Programação orientada a objetos

- O paradigma envolve herança:
  - as **variáveis polimórficas** da superclasse podem referenciar objetos de qualquer uma de suas subclasses

# Programação orientada a objetos

- O paradigma envolve herança:
  - as **variáveis polimórficas** da superclasse podem referenciar objetos de qualquer uma de suas subclasses
  - qual implementação de método deve ser executada? Isso é definido muitas vezes via vinculação tardia (*late binding*).

# Programação orientada a objetos

```
public static void main(String[] args) {
    Animal a;
    a = new Gato();
    a.falar(); // decide que chamará implementação de Gato em runtime
    a = new Cachorro();
    a.falar(); // decide que chamará implementação de Cachorro em runtime
    List<Animal> animais = new ArrayList<Animal>();
    animais.add(new Gato());
    animais.add(new Gato());
    animais.add(new Cachorro());
    for(Animal pet : animais) { pet.falar(); } // decide em runtime que impl. chamar
}
```

# Programação orientada a objetos

Em Java, também existe vinculação estática (*static binding*) (e.g. métodos estáticos)

# Programação orientada a objetos

- Em linguagens OO, existe o conceito de que “tudo vira objeto” ou o de que “tudo é ou modelo de tipo imperativa ou objeto”.

# Programação orientada a objetos

- Em linguagens OO, existe o conceito de que “tudo vira objeto” ou o de que “tudo é ou modelo de tipo imperativa ou objeto”.

- No segundo caso, utiliza-se o conceito de classes envoltórias/empacotadoras (*wrappers*), que é o “tipo objeto” referente a um tipo primitivo (e.g. Java tem int, que é o tipo primitivo, e Integer, que é a classe empacotadora).

# Exemplo de linguagem OO: Java

- Linguagem projetada pela Sun Microsystems nos anos 90.

# Exemplo de linguagem OO: Java

- Linguagem projetada pela Sun Microsystems nos anos 90.

- É uma linguagem OO mais pura que C++. Em Java, programas são classes e todos os valores compostos são objetos.

# Exemplo de linguagem OO: Java

- Linguagem projetada pela Sun Microsystems nos anos 90.
- É uma linguagem OO mais pura que C++. Em Java, programas são classes e todos os valores compostos são objetos.
- É uma linguagem que dá suporte para programação concorrente e programação distribuída.

# Exemplo de linguagem OO: Java

- Algumas características de Java
  - os valores compostos são objetos (e.g. arrays)
  - permite classes recursivas
  - linguagem estaticamente tipada

# Exemplo de linguagem OO: Java

```java
public class Node {

    private int value;
    // Em Node, há um atributo que é um Node (ele mesmo)
    private Node next;
```

# Exemplo de linguagem OO: Java

- Algumas características de Java
  - as variáveis em Java podem ser locais (objetos) ou globais (variáveis de classe), bem como na heap (objetos, automaticamente destruídos);
  - possui expressões condicionais mas não iterativas;
  - possui alguns sequenciadores de C++, mas não possui o desvio incondicional.

# Exemplo de linguagem OO: Java

- Algumas características de Java
  - as exceções são objetos que herdam de uma classe chamada Exception;
  - o programa principal é um que possui um método chamado main();
  - `final` utilizado para constantes, métodos que não são sobrescritos ou classes que não são herdadas.

# Exemplo de linguagem OO: Java

- Algumas características de Java
  - há distinção de variáveis e métodos de instância ou de classe através do `static`.
  - possui os modificadores de visibilidade `public`, `private` e `protected`;
  - permite polimorfismo via herança (e também interface).

# Exemplo de linguagem OO: Java

- Algumas características de Java
  - o conceito de *late binding*, onde as classes são vinculadas ao programa em tempo de execução e apenas se forem necessárias.

# Programação orientada a objetos

- Nesse paradigma, o código pode ser bastante organizado, reutilizado, com tempo reduzido para manutenção, bem como esse paradigma apresenta grande utilização comercial.

# Programação orientada a objetos

- Nesse paradigma, o código pode ser bastante organizado, reutilizado, com tempo reduzido para manutenção, bem como esse paradigma apresenta grande utilização comercial.

- A desvantagem está na eficiência do código, que costuma ser menor.

# Programação orientada a eventos

- Nesse paradigma, os programas não controlam a sequência dos eventos de entrada, o que vai na contramão do que ocorre em modelos convencionais de programa.

# Programação orientada a eventos

- Nesse paradigma, os programas não controlam a sequência dos eventos de entrada, o que vai na contramão do que ocorre em modelos convencionais de programa.

- O programa irá reagir a alguma sequência de eventos que venha a acontecer.

# Programação orientada a eventos

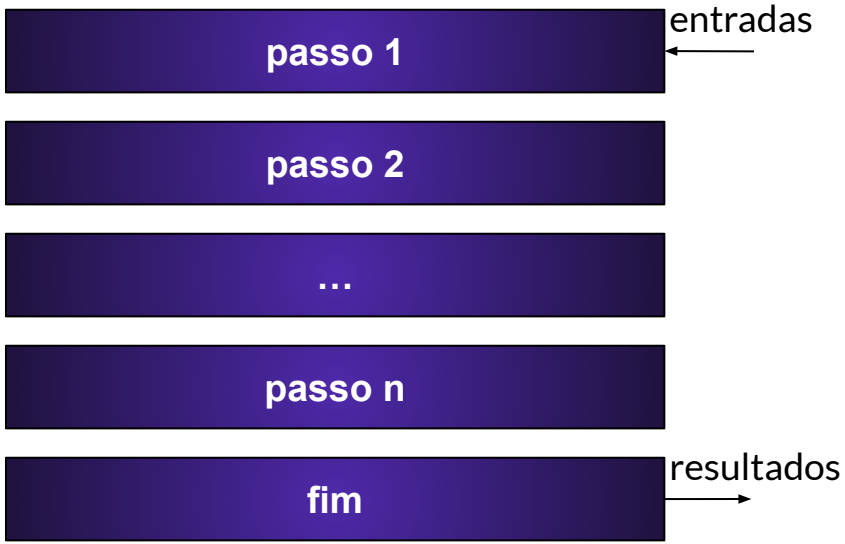
- Nesse paradigma, os programas não controlam a sequência dos eventos de entrada, o que vai na contramão do que ocorre em modelos convencionais de programa.

- O programa irá reagir a alguma sequência de eventos que venha a acontecer.

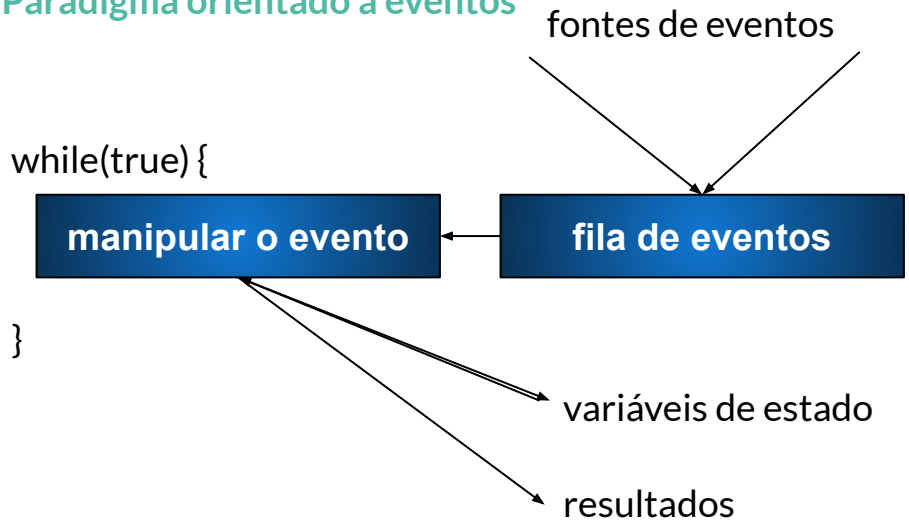
- Os programas com GUI (*graphical user interface*) são exemplos comuns onde isso acontece.

# Programação orientada a eventos

## Paradigma imperativo




## Paradigma orientado a eventos



# Programação orientada a eventos

- Os programas orientados a eventos não têm um ponto de término pré-determinado.

# Programação orientada a eventos

- Os programas orientados a eventos não têm um ponto de término pré-determinado.

- Leitura, avaliação e impressão podem não acontecer nessa ordem (diferente de programas mais convencionais, onde essa ordem é seguida).

# Programação orientada a eventos

- Os programas orientados a eventos não têm um ponto de término pré-determinado.

- Leitura, avaliação e impressão podem não acontecer nessa ordem (diferente de programas mais convencionais, onde essa ordem é seguida).

- Exemplos de aplicações: jogos, caixa eletrônico.

# Programação orientada a eventos

- Os programas orientados a eventos não têm um ponto de término pré-determinado.
- Leitura, avaliação e impressão podem não acontecer nessa ordem (diferente de programas mais convencionais, onde essa ordem é seguida).
- Exemplos de aplicações: jogos, caixa eletrônico.
- É comum a utilização do modelo MVC p/ isso.

# Referências

SEBESTA, Robert M. **Conceitos de Linguagens de Programação**. Bookman, 2018.